Automated E-Commerce Deployment Platform

Project Documentation – Part 1: Planning & Management, Part 2: Literature Review, Part 3: Requirements Gathering

Program: DEPI Graduation Project | Repository: https://github.com/alimohamed22233aaa-ai/Depi_Graduation_Project

Team members: Ahmed Mahmoud, Ali Mohamed, Mohamed Hassan, Nouran Salm, Marwan Esam, Abdelrahman Owais

1. Project Planning & Management

1.1 Project Idea

E-commerce websites lose revenue whenever they go down, and releasing new versions of a multi-service shop is usually manual, slow and risky. The project tackles this by deploying a multi-service e-commerce platform (web, database, payment and search) through a fully automated DevOps lifecycle: containerization, CI/CD with security scanning, Kubernetes orchestration with auto-scaling, infrastructure provisioning on AWS, monitoring and automated rollback.

The focus of the project is **the delivery platform, not the shop itself**. The application is an existing open-source microservices shop (Google's Online Boutique), extended by the team with two services (search and orders) so that every required component exists. Everything around the application (pipelines, manifests, infrastructure code, configuration and dashboards) is written by the team. The deployment files that ship with the source project are specific to Google Cloud and were removed and replaced by the team's own.

1.2 Objectives

1. Containerize every microservice with reproducible, optimized Docker images.
2. Build a CI/CD pipeline that lints, tests, scans and publishes images, then deploys them automatically.
3. Run the platform on Kubernetes with zero-downtime rolling updates and CPU-based auto-scaling.
4. Provision the AWS infrastructure (EC2, RDS, S3, ELB) entirely as code with Terraform.
5. Configure servers automatically with Ansible.
6. Route traffic through an Nginx reverse proxy / ingress.
7. Monitor every service with Prometheus and Grafana dashboards.
8. Detect failed releases and roll back automatically.
9. Keep all code and scripts in a single Git repository.

1.3 Scope

In scope: the deliverables above, the two added services (search and orders), the security checks in the pipeline (secret, code, infrastructure and image scanning), and the documentation and final presentation.

Out of scope: writing new shop features or a custom frontend, real payment processing (the payment service is a mock), multi-region deployment, and production-grade cost optimization.

1.4 Architecture Overview

Users reach the platform through an AWS load balancer (ELB) that forwards traffic to an Nginx ingress inside Kubernetes. The ingress routes requests to the frontend, the search service and the orders service. The frontend calls the product catalog, cart, recommendation, shipping, currency, ad and checkout services over gRPC. The checkout service calls payment, shipping and email, and records each completed order in the orders service, which stores it in PostgreSQL on Amazon RDS. The cart uses Redis. Prometheus scrapes metrics from the services and Grafana visualizes them. S3 is used for Terraform remote state and backups.

1.5 Team and Responsibilities

Team Member	Domain	Main Technical Responsibility	Primary Deliverable
Ahmed Mahmoud	Ansible	Configuration management, host automation and package setup	Playbooks, dynamic inventories, systemd rules
Ali Mohamed	CI/CD Pipeline	Automated build, test verification, image publishing and deployment trigger	Jenkinsfile / GitHub Actions pipeline code
Mohamed Hassan	Kubernetes	Orchestration coordinator, pod scaling, ingress routing and deployment manifests	K8s Deployment/Service manifests, HPA configs
Nouran Salm	Terraform	Infrastructure as Code provisioning for AWS cloud resources	Terraform modules (EC2, RDS, S3, ELB)
Marwan Esam	Docker	Microservice containerization, multi-stage builds and image optimization	Dockerfile specs, compose files, optimized images
Abdelrahman Owais	Monitoring & Lead	Prometheus/Grafana observability setup and team presentation coordination	Grafana dashboards, scrape configs, presentation

Every member also reviews pull requests in the neighbouring domain (for example, Docker reviews CI changes), so knowledge is shared and no domain depends on one person.

1.6 Tools and Technologies

Git and GitHub (source control), Docker and Docker Compose (containers), GitHub Actions, Gitleaks, SonarQube, Checkov and Trivy (security and quality scanning), Kubernetes with Horizontal Pod Autoscaler (orchestration), Nginx (reverse proxy / ingress), Terraform (infrastructure), Ansible (configuration), AWS EC2, RDS, S3 and ELB (cloud), Prometheus and Grafana (monitoring).

1.7 Work Plan

The plan below fits a 4-week schedule. Phases overlap, so every member starts working in week 1.

Phase	Timing	Activities	Owner(s)
1. Setup and documentation	Week 1	Repository structure, application code, documentation parts 1–3	All
2. Containerization	Weeks 1–2	Dockerfiles for all services, compose file for local run, image optimization	Docker
3. CI pipeline	Weeks 1–3	Lint, tests, secret/code/IaC/image scanning, build and push	CI/CD
4. Kubernetes	Weeks 2–3	Deployments, Services, ingress, HPA, rolling update strategy, probes	Kubernetes
5. Infrastructure	Weeks 2–3	Terraform modules for EC2, RDS, S3 and ELB; Ansible roles for server setup	Terraform, Ansible

Phase	Timing	Activities	Owner(s)
6. Delivery and rollback	Week 3	Automated deployment from the pipeline, health verification, automatic rollback	CI/CD, Kubernetes
7. Monitoring	Weeks 3–4	Prometheus scraping, Grafana dashboards, alerts	Monitoring
8. Integration and testing	Week 4	End-to-end tests, load test to demonstrate auto-scaling, failure and rollback drills	All
9. Final delivery	Week 4	Final documentation, demo rehearsal, presentation	Lead, All

Collaboration rules: one `main` branch protected by pull requests, short-lived feature branches, at least one reviewer per merge, a weekly team meeting, and a short written status update per member each week.

1.8 Risks and Mitigation

Risk	Impact	Mitigation
AWS cost overrun	Budget loss	Use small instance sizes, destroy resources after each session (`terraform destroy`), set AWS budget alerts
Services written in five languages are hard to containerize	Delays in Docker and CI	Split Dockerfiles across the team, start with a few services, reuse build hints in the service reference
Resource limits on laptops when running the whole stack	Slow development	Run a reduced set of services locally, use a shared AWS environment for full tests
Security scanners block the pipeline with many findings	Pipeline stuck	Start in report-only mode, fix findings gradually, then enforce failure on high and critical issues
A failed release breaks the shop	Downtime during demo	Rolling update with readiness probes, automatic rollback, rehearsed failure drill
Dependency on one member's domain	Single point of failure	Cross-reviews, documented runbooks, shared repository
Secrets committed by mistake	Security incident	Gitleaks in the pipeline, secrets stored in GitHub Secrets / Kubernetes Secrets

1.9 Success Metrics (KPIs)

The targets below are team goals to be measured and reported in the final presentation.

Metric	Target
Pipeline duration from commit to deployed	under 15 minutes
Downtime during a rolling update	zero failed requests during the update in the load test
Time to automatic rollback after a failed release	under 3 minutes
Auto-scaling	pods increase under load and return to the minimum afterwards
Pipeline gates	no high or critical findings in a release image
Infrastructure reproducibility	full AWS environment created from scratch with one Terraform run
Monitoring coverage	every service visible in Grafana with request rate, errors and latency

2. Literature Review

This section compares this project with existing open-source microservices demonstrations. The facts below are summarized from the projects' public repositories; the team should re-check each source and add the access date before submission.

2.1 Existing Projects

Online Boutique (Google Cloud Platform). An e-commerce demo made of about eleven microservices written in several languages that communicate over gRPC. It includes a frontend, product catalog, cart, checkout, payment, shipping, currency, email, recommendation and ad services, plus a load generator. It ships with Kubernetes manifests and is designed to demonstrate Google Cloud and Kubernetes features.

OpenTelemetry Demo (Astronomy Shop). A larger microservices shop maintained by the OpenTelemetry community to demonstrate distributed tracing, metrics and logs across many languages. It bundles its own observability stack. Source: https://github.com/open-telemetry/opentelemetry-demo

Other references. The team may also review Sock Shop (a microservices reference application from Weaveworks) and Microsoft's eShopOnContainers, which are commonly used to teach container and orchestration concepts. These should be cited with their current repository links and status.

2.2 Strengths and Limitations

Project	Strengths	Limitations for this project
Online Boutique	Realistic polyglot shop, small enough to run, widely known	Deployment tooling targets Google Cloud; no search service; no relational database; no AWS infrastructure code
OpenTelemetry Demo	Rich observability, many services	Heavy to run, built-in monitoring overlaps with the required Prometheus/Grafana work, not focused on delivery pipelines
Teaching shops (Sock Shop, eShopOnContainers)	Good learning material	Several are older or less actively maintained; not tied to a full CI/CD, IaC and rollback workflow

2.3 How This Project Differs

The existing projects are mainly **applications for demonstrating a platform or a feature**. This project treats the application as given and builds the **delivery system around it**:

- an AWS-based infrastructure defined in Terraform and configured with Ansible, instead of a single cloud vendor's tooling;
- a CI/CD pipeline that includes secret scanning, static analysis, infrastructure-as-code scanning and image vulnerability scanning before anything is deployed;
- zero-downtime rolling updates, auto-scaling and automatic rollback verified by failure drills;
- dedicated Prometheus and Grafana dashboards instead of a bundled stack;
- two added services (search and orders with an RDS-backed database) so the platform covers web, database, payment and search as required.

2.3.1 Ideas Reused

The team reuses the application code and the service communication model of Online Boutique under its open-source license, and follows the common practices these projects demonstrate (health endpoints, metrics endpoints, stateless services). All deployment, infrastructure and automation work is original to the team.

3. Requirements Gathering

3.1 Stakeholders

Stakeholder	Needs
Shop customers	A fast, always-available store where browsing, search and checkout work reliably
Shop owner / business	Few outages, quick release of new features, controlled cloud cost
Developers	Fast feedback from automated tests and scans, simple and repeatable releases
Operations / DevOps engineers	Reproducible infrastructure, visibility into service health, safe rollback
Security reviewers	Evidence that code, secrets, configurations and images are checked before release
Project supervisors / evaluators	A working, documented demonstration of every required deliverable

3.2 Functional Requirements

ID	Requirement
FR-1	Users can browse products and view product details.
FR-2	Users can search products by keyword and category.
FR-3	Users can add items to a cart and complete checkout.
FR-4	Payment is processed through a payment service (mock gateway).
FR-5	Each completed order is stored in a relational database (PostgreSQL on RDS) and can be retrieved through an API.
FR-6	Every service is packaged as a Docker image.
FR-7	A push to the repository triggers the pipeline: lint, tests, security scans, image build and publish.
FR-8	A successful pipeline deploys the new version to Kubernetes with a rolling update.
FR-9	The deployment is verified, and if it fails the previous version is restored automatically.
FR-10	Kubernetes scales services up and down automatically according to CPU load.
FR-11	Requests are routed through an Nginx reverse proxy / ingress to the correct service.
FR-12	AWS resources (EC2, RDS, S3, ELB) are created and destroyed by Terraform.
FR-13	Servers are configured automatically with Ansible.
FR-14	Prometheus collects metrics from all services and Grafana displays dashboards for them.
FR-15	All code and scripts are versioned in one Git repository.

3.3 Non-Functional Requirements

Category	Requirement
Availability	Releases must not interrupt service (rolling update with readiness checks, no pod removed before its replacement is ready).
Scalability	Frequently loaded services scale horizontally between a defined minimum and maximum number of pods.
Performance	The store remains responsive under the load-generator traffic used in the demo.
Security	No secrets in the repository, containers run as non-root, images and infrastructure code are scanned, database access is restricted to the application.
Reliability	Health endpoints on every service; failed releases are reverted automatically.
Observability	Request rate, error rate, latency and resource usage are visible per service.
Maintainability	Infrastructure and configuration are code, reviewed through pull requests and documented.
Portability	The platform runs locally with Docker Compose or a local Kubernetes cluster, and on AWS with the same images.
Cost	Cloud resources are small and destroyed when not in use.
Usability	A single documented procedure to build, deploy and roll back.

3.4 Constraints and Assumptions

- The required toolset is fixed by the project brief (Docker, Kubernetes, Prometheus, Grafana, Ansible, Terraform, AWS EC2/RDS/S3/ELB, Nginx, Git, and a CI/CD server).
- The application is based on an open-source project used under its license; its source is credited in all documents.
- Payment is simulated; no real card data is processed.
- The team works with limited AWS budget and student hardware.

3.5 Acceptance Criteria

The project is considered complete when the team can demonstrate, in order: a code change triggers the pipeline; scans and tests pass or block as designed; the new image is deployed without downtime; a deliberately broken release is rolled back automatically; a load test makes the pods scale up and then down; Grafana shows the behaviour; and the whole AWS environment can be rebuilt from the repository.